

Transformers.

Transfer learning $\rightarrow$ multimodal task $\rightarrow$ NLP + Image

* In Attention mechanism the drawback or the disadvantage is, we can't give the parallel seq all the words in a sentence $\rightarrow$ Not Scalable.

Dataset $\rightarrow$ huge $\rightarrow$ Scalable with respect to training.

Transformers $\neq$ LSTM RNN

$\rightarrow$ contains self-attention module
 $\rightarrow$ All the words will parallelly sent to encoder.

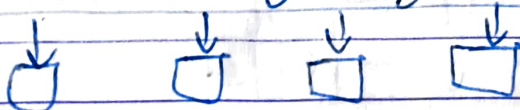
$\Downarrow$
Positional Encoding.

Transformer $\rightarrow$ $\uparrow\uparrow\uparrow$ Dataset $\uparrow\uparrow\uparrow$ $\rightarrow$ Amazing
SOTA $\leftarrow$ NLP

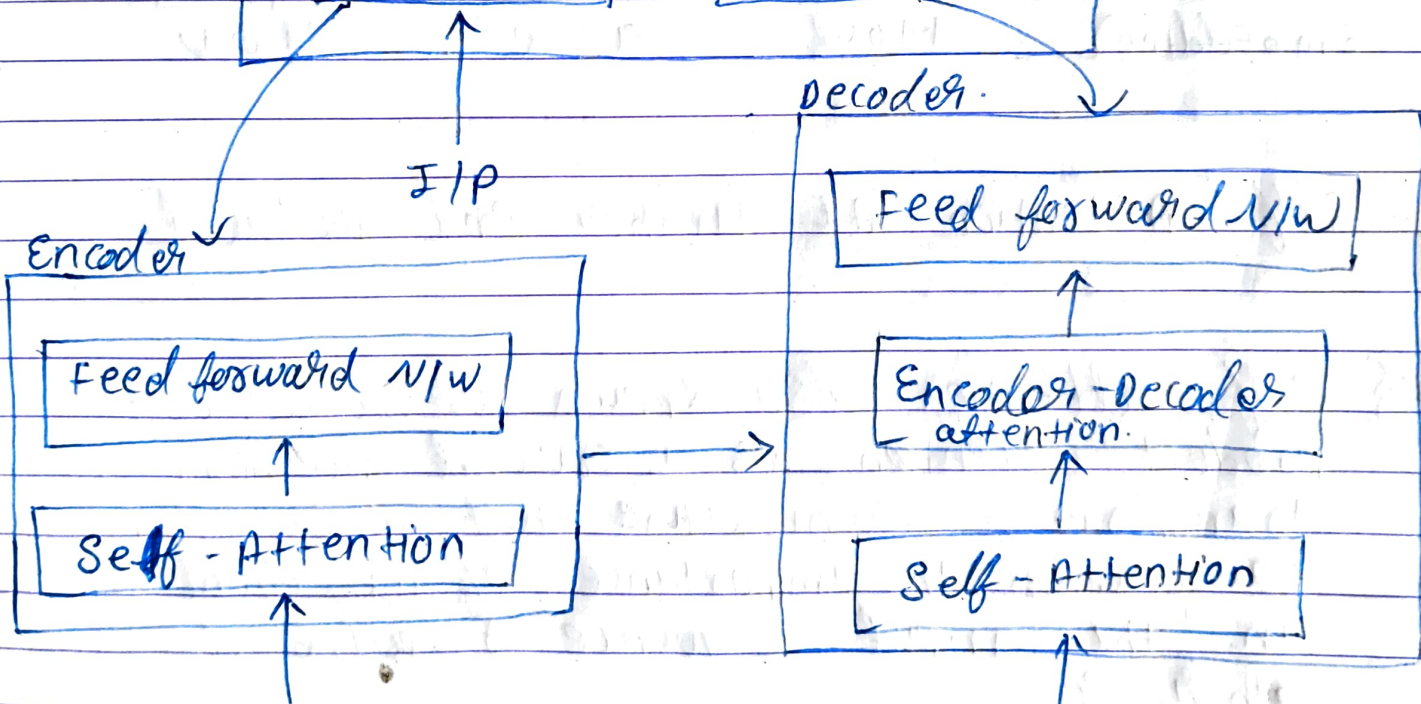
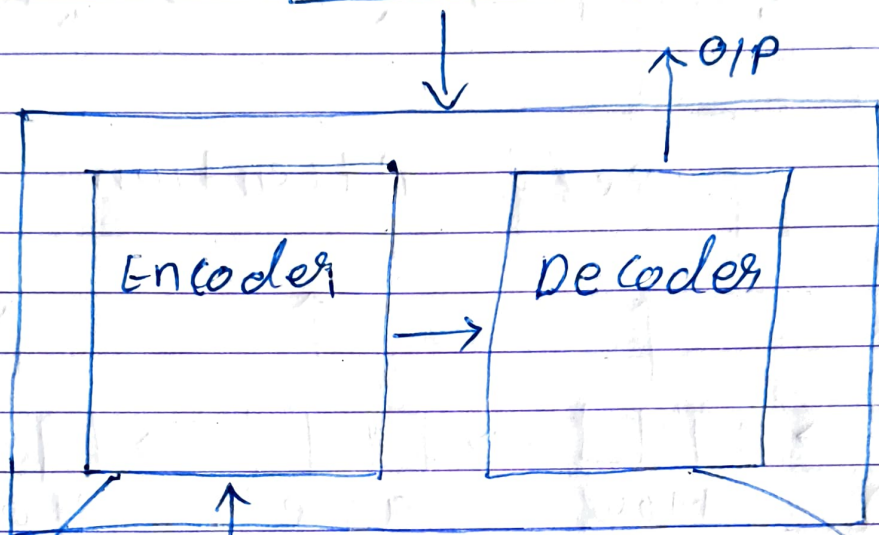
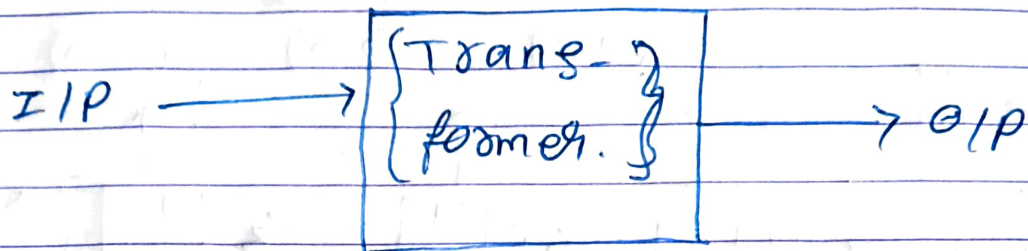
Contextual Embedding $\Rightarrow$ related.

my name is vidya, I play cricket.

$\downarrow$
[Embedding Layer]

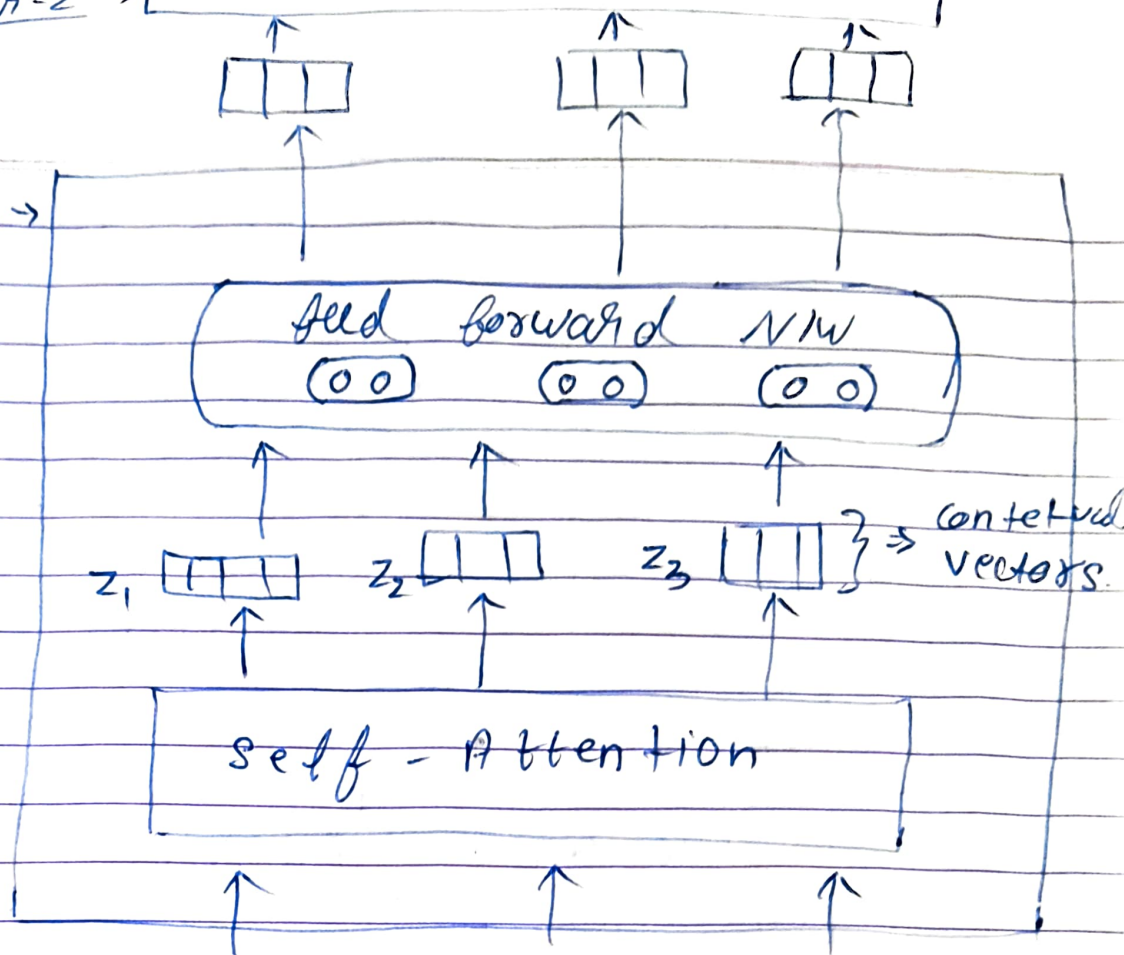


Basic Architecture of Transformer



Encoder-2 $\rightarrow$ L

Encoder-1 $\rightarrow$



Vectors $\rightarrow$ $\begin{bmatrix} \square & \square & \square \end{bmatrix} \rightarrow \begin{bmatrix} \square & \square & \square \end{bmatrix} \rightarrow \begin{bmatrix} \square & \square & \square \end{bmatrix}$
 (Embedding) How are you

$\rightarrow$ Self Attention At Higher And Detailed.

$\Rightarrow$ { Self-attention, also known as scaled dot-product attention, is a crucial mechanism in the transformer architecture that allows the model to weigh the importance of different tokens in the input sequence relative to each other. }

~~Idea~~ Input $\Rightarrow$ Queries (Q), Keys (K), values (V).

① Query vectors (Q):

It represents the token for which we are calculating the attention. They help determine the importance of other tokens in the context of the current token.

Importance $\Rightarrow$

- ① focus determination.
- ② contextual understanding.

② Key vectors (K):

This represents all the tokens in the sequence and are used to compare with the query to calculate attention scores.

Importance $\Rightarrow$

- ① Relevance measurement
- ② Information Retrieval.

③ value vectors (V):

It is held the actual information that will be aggregated to form the output of the ~~calculation~~ attention mechanism.

Importance $\Rightarrow$

- ① Information Aggregation.
- ② Context preservation.

Input Sequence = ["The", "cat", "sat"]

Embedding size = 4 , $Q, K, V \Rightarrow 4$

① Token Embedding $\Rightarrow$

$$E_{\text{The}} \Rightarrow [1 \ 0 \ 1 \ 0]$$

$$E_{\text{cat}} \Rightarrow [0 \ 1 \ 0 \ 1]$$

$$E_{\text{sat}} \Rightarrow [1 \ 1 \ 1 \ 1]$$

② Linear Transformation $\Rightarrow$

we create Q, K, V by multiplying the embeddings by learned weights matrices w_Q, w_K, w_V .

$$w_Q = w_K = w_V \Rightarrow I \Rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$Q_{\text{The}} \Rightarrow [1 \ 0 \ 1 \ 0] \cdot w_Q \Rightarrow [1 \ 0 \ 1 \ 0]$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$K_{\text{The}} = [1 \ 0 \ 1 \ 0] \cdot w_K = [1 \ 0 \ 1 \ 0]$$

$$V_{\text{The}} = [1 \ 0 \ 1 \ 0] \cdot w_V = [1 \ 0 \ 1 \ 0]$$

$$\textcircled{i} Q_{\text{The}} = K_{\text{The}} = V_{\text{The}} \Rightarrow [1, 0, 1, 0]$$

$$\textcircled{ii} Q_{\text{cat}} = K_{\text{cat}} = V_{\text{cat}} \Rightarrow [0, 1, 0, 1]$$

$$\textcircled{iii} Q_{\text{sat}} = K_{\text{sat}} = V_{\text{sat}} \Rightarrow [1, 1, 1, 1]$$

③ Computing Attention Scores: $\Rightarrow$

$$\textcircled{i} \underset{\substack{\downarrow \\ \text{token}}}{\text{The}} \Rightarrow \text{score}(Q_{\text{The}}, K_{\text{The}}) = [1 \ 0 \ 1 \ 0] \cdot [1 \ 0 \ 1 \ 0]^T$$

$$\Rightarrow [1 \ 0 \ 1 \ 0] \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} = 2$$

$$\text{score}(Q_{\text{The}}, K_{\text{cat}}) = [1 \ 0 \ 1 \ 0] \cdot [0 \ 1 \ 0 \ 1]^T \Rightarrow 0$$

$$\text{score}(Q_{\text{The}}, K_{\text{sat}}) = [1 \ 0 \ 1 \ 0] \cdot [1 \ 1 \ 1 \ 1]^T = 2$$

② cat $\rightarrow$ Token

$$\text{score}(Q_{\text{cat}}, K_{\text{The}}) = [0 \ 1 \ 0 \ 1] \cdot [1 \ 0 \ 1 \ 0]^T = 0$$

$$\text{score}(Q_{\text{cat}}, K_{\text{cat}}) = [0 \ 1 \ 0 \ 1] \cdot [0 \ 1 \ 0 \ 1]^T = 2$$

$$\text{score}(Q_{\text{cat}}, K_{\text{sat}}) = [0 \ 1 \ 0 \ 1] \cdot [1 \ 1 \ 1 \ 1]^T = 2$$

iii) for the token sat.

$$\text{Score}(Q_{\text{sat}}, K_{\text{re}}) = [1 \ 1 \ 1 \ 1] \cdot [1 \ 0 \ 1 \ 0]^T = 2$$

$$\text{Score}(Q_{\text{sat}}, K_{\text{cat}}) = [1 \ 1 \ 1 \ 1] \cdot [0 \ 1 \ 0 \ 1]^T = 2$$

$$\text{Score}(Q_{\text{sat}}, K_{\text{sat}}) = [1 \ 1 \ 1 \ 1] \cdot [1 \ 1 \ 1 \ 1] = 4$$

iv) Scaling $\rightarrow$

we take up the scores and scale down by dividing the scores by the $\sqrt{(\text{dim})_K}$

$$\Rightarrow \sqrt{(\text{dim})_K} = \sqrt{(K)_{\text{dim}}} \Rightarrow \sqrt{d_K} = \sqrt{4} = 2$$

scaling in the attention mechanism is crucial to prevent the dot product from growing too large $\Rightarrow$

It ensures stable gradients during training

d_K is large $\rightarrow$ ① Gradient Exploding.

two problems we are facing $\rightarrow$ ② softmax saturation.

Let $\Rightarrow$

$$Q = [2 \ 3 \ 4 \ 1] \quad , \quad K_1 = [1 \ 0 \ 1 \ 0]$$

$$K_2 = [0 \ 1 \ 0 \ 1]$$

without scaling.

$$Q \cdot K_1^T = 2 \times 1 + 3 \times 0 + 4 \times 1 + 1 \times 0 \Rightarrow 2 + 4 = \underline{6}$$

$$Q \cdot K_2^T = 2 \times 0 + 3 \times 1 + 4 \times 0 + 1 \times 1 \Rightarrow 0 + 3 + 0 + 1 = 4$$

* Scores $[6, 4] \Rightarrow$ without scaling.

applying softmax. ($[6, 4]$)

$$\Rightarrow \left[\frac{e^6}{e^6 + e^4}, \frac{e^4}{e^6 + e^4} \right]$$

$$\Rightarrow \left[\frac{e^6}{e^6(1+e^{-2})}, \frac{e^4}{e^4(1+e^2)} \right]$$

$$\Rightarrow \left[\frac{1}{(1+e^{-2})}, \frac{1}{e^2+1} \right] \Rightarrow \approx [0.88, 0.12]$$

$\Rightarrow$ most of the attention weight is assigned to the first Key vector, very little to the second vector,

$\Rightarrow$ with scaling.

① compute scaled dot product.

$$[6, 4] \Rightarrow \text{Scaled} \Rightarrow \left[\frac{6}{\sqrt{d_K}}, \frac{4}{\sqrt{d_K}} \right]$$

d_K = dimension of (K)

$$\sqrt{d_K} = \sqrt{4} = 2$$

$$[6, 4] \Rightarrow \left[\frac{6}{2}, \frac{4}{2} \right] = [3, 2]$$

$$\text{Softmax} \Rightarrow \left[\frac{e^3}{e^3 + e^2}, \frac{e^2}{e^3 + e^2} \right]$$

$$\Rightarrow \left[\frac{e^3}{e^3(1+e^{-1})}, \frac{e^2}{e^2(e+1)} \right]$$

$$\Rightarrow \left[\frac{1}{1+e^{-1}}, \frac{1}{e+1} \right] \approx [0.73, 0.27]$$

$\Downarrow$
attention weights.

{ Here, the attention weights are more balanced compared to the unscaled case.

Imp Scaling ensures that the dot products are kept within a range that allows the softmax function to operate effectively, providing a more balanced distribution of attention weight and improving the overall learning process the model.

~~Score~~ - Score for "The" $\Rightarrow [1, 0, 1]$

~~Score~~ - Score for "cat" $\Rightarrow [0, 2, 2]$

~~Score~~ - Score for "sat" $\Rightarrow [2, 2, 4]$

⑤ Apply softmax.

Attention weight "The" = $\text{Softmax}([1, 0, 1])$

$$\Rightarrow [0.4223, 0.1554, 0.4223]$$

Attention weight "cat" = $\text{softmax}([0, 2, 2])$

$$\Rightarrow [0.1554, 0.4223, 0.4223]$$

Attention weight "sat" $\Rightarrow \text{Softmax}([2, 2, 4])$

$$\Rightarrow [0.2119, 0.2119, 0.5762]$$

⑥ weight sum of values $\Rightarrow$

we multiply the attention weight by corresponding value vectors. ~~for the token~~

For the Token "The" =

$$= (0.4223 * V_{\text{The}}) + (0.1554 * V_{\text{cat}})$$

$$\Rightarrow \{0.4223 \cdot [1 \ 0 \ 1 \ 0] + 0.1554 \cdot [0 \ 1 \ 0 \ 1] + 0.4223 \cdot [1 \ 1 \ 1 \ 1]\}$$

$$\Rightarrow [0.4223, 0, 0.4223, 0] + [0, 0.1554, 0, 0.1554] + [0.4223, 0.4223, 0.4223, 0.4223]$$

$$\Rightarrow [1.2669, 0.999, 1.2669, 0.999]$$

The $\rightarrow [1 \ 0 \ 1 \ 0] \rightarrow$ embedding vector

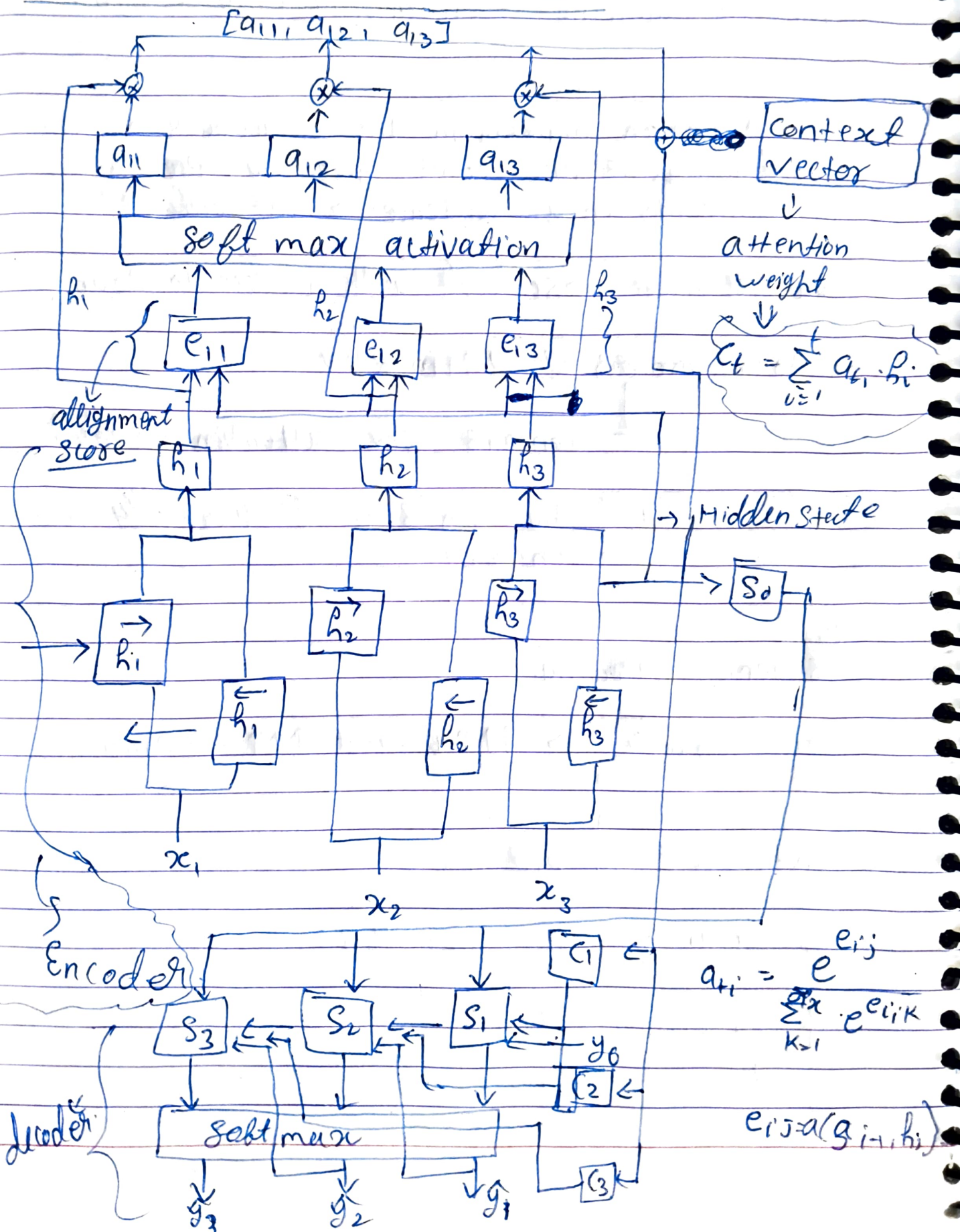
$\downarrow$
after the self attention model or the layer the value becomes.

$\downarrow$
[1.2669, 0.9999, 1.2669, 0.9999]
 $\rightarrow$ Known as the contextual vector

$\rightarrow$ process for finding the contextual vector.

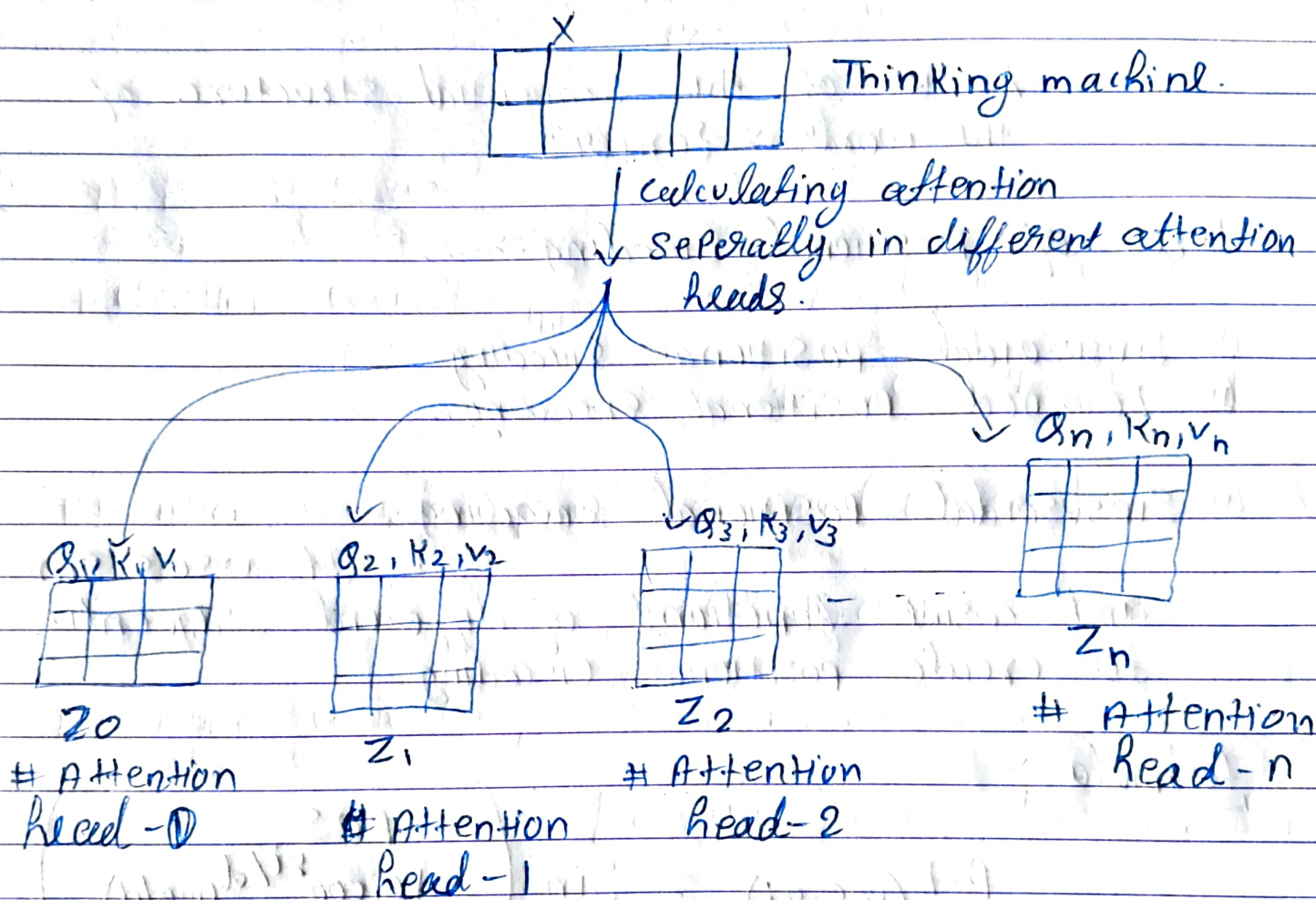
- (i) take the embedded vector
- (ii) Initialize Q, K, V, W_Q, W_K, W_V
- (iii) finding the Attention score
- (iv) scaled the score
- (v) apply the softmax and find the values.
- (vi) calculate the weighted sum of value ($\text{softmax} \times V_i$)

Sequence to Sequence mechanism.

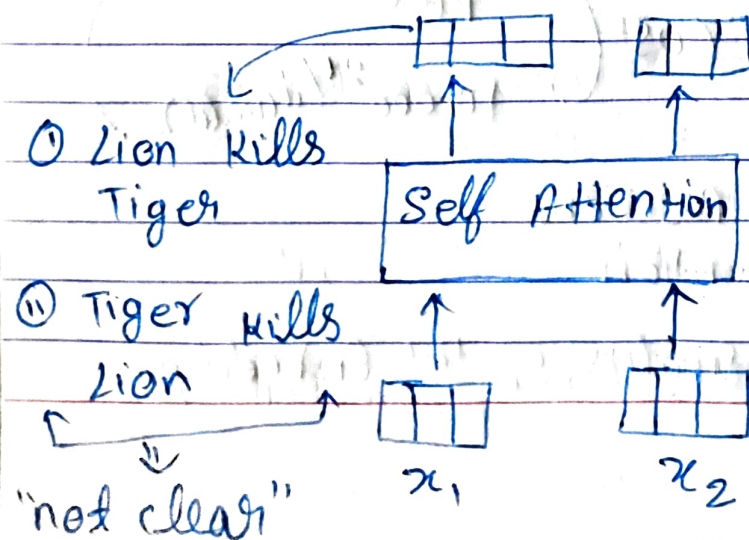


Multi Head Attention

It expands model's ability to focus on different position of tokens.



Positional Encoding $\Rightarrow$ Representing order of sequence.



Advantage: In the transformer the words Tokens are process parallelly



draw back.



Lack of the sequential structure of the words $\Rightarrow$ {order}

Types of Positional Encoding $\Rightarrow$

- ① Sinsusoidal positional Encoding.
- ② Learned Positional Encoding.

① Sinsusoidal positional Encoding $\Rightarrow$

It uses sine and cosine functions of different frequencies to create positional encoding.

formula $\Rightarrow$

$$P.E(pos, 2i) = \sin\left(\frac{pos}{10,000^{2i/d(model)}}\right)$$

$$P.E(pos, 2i+1) = \cos\left(\frac{pos}{10,000^{2i/d(model)}}\right)$$

$\Rightarrow$ where, pos is the position
i is the dimension

$d(model)$ is the dimensionality of the Embeddings.

Eg → "The cat sat"

The → [0.1 0.2 0.3 0.4]

cat → [0.5 0.6 0.7 0.8]

Sat → [0.9 1.0 1.1 1.2]

→ Embedding vectors.

$$P.E(0,0) = \sin\left(\frac{0}{10,000^{0.14}}\right) = 0$$

$$P.E(0,1) = \cos\left(\frac{0}{10,000^{0.14}}\right) = 1$$

$$P.E(0,2) = \sin\left(\frac{0}{10,000^{0.214}}\right) = 0$$

$$P.E(0,3) = \cos\left(\frac{0}{10,000^{0.314}}\right) = 1$$

$$P.E \Rightarrow [0 \ 1 \ 0 \ 1]$$

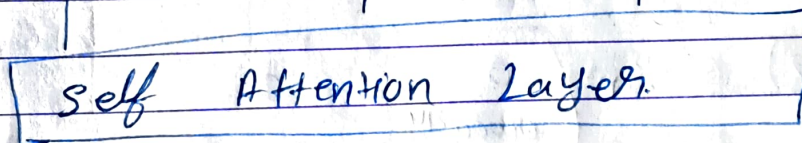
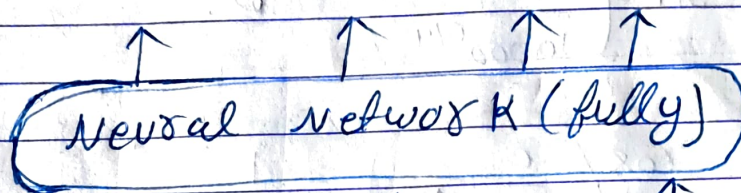
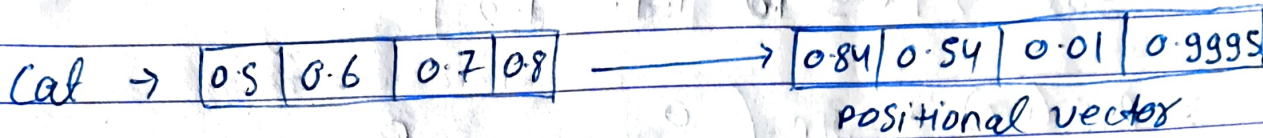
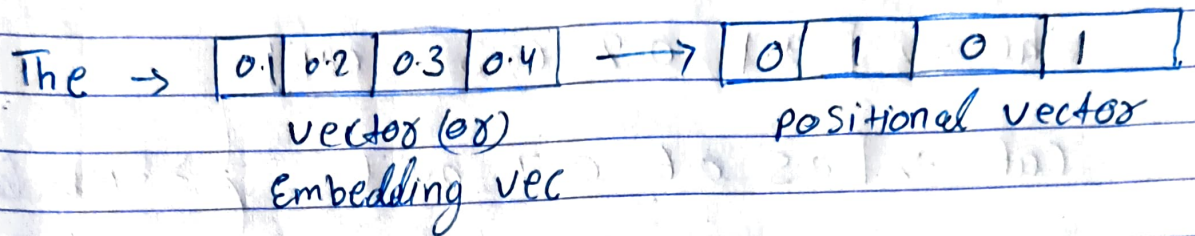
for "cat"

$$P.E(1,0) = \sin\left(\frac{1}{10,000^{0.14}}\right) = \sin(1) = 0.8415$$

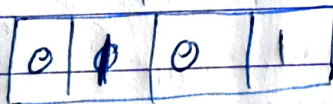
$$P.E(1,1) = \cos\left(\frac{1}{10,000^{0.214}}\right) \approx 0.5403$$

$$P.E(1,2) = \sin\left(\frac{1}{10,000^{0.214}}\right) = 0.01$$

$$P-E(1,3) = \log\left(\frac{1}{10,000^{3/4}}\right) \approx 0.99995$$

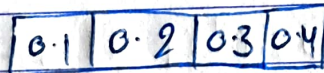


positional
encoding.

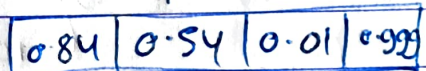


+

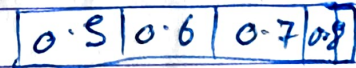
vector
Embedding



The

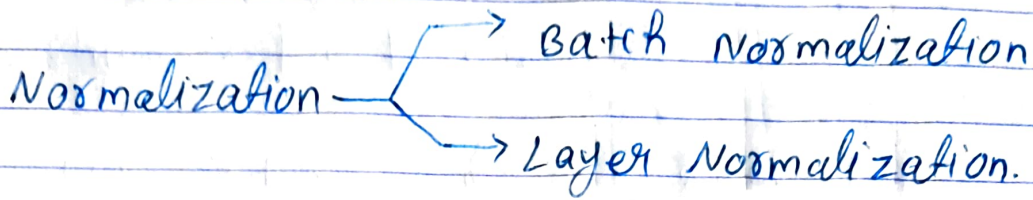


+

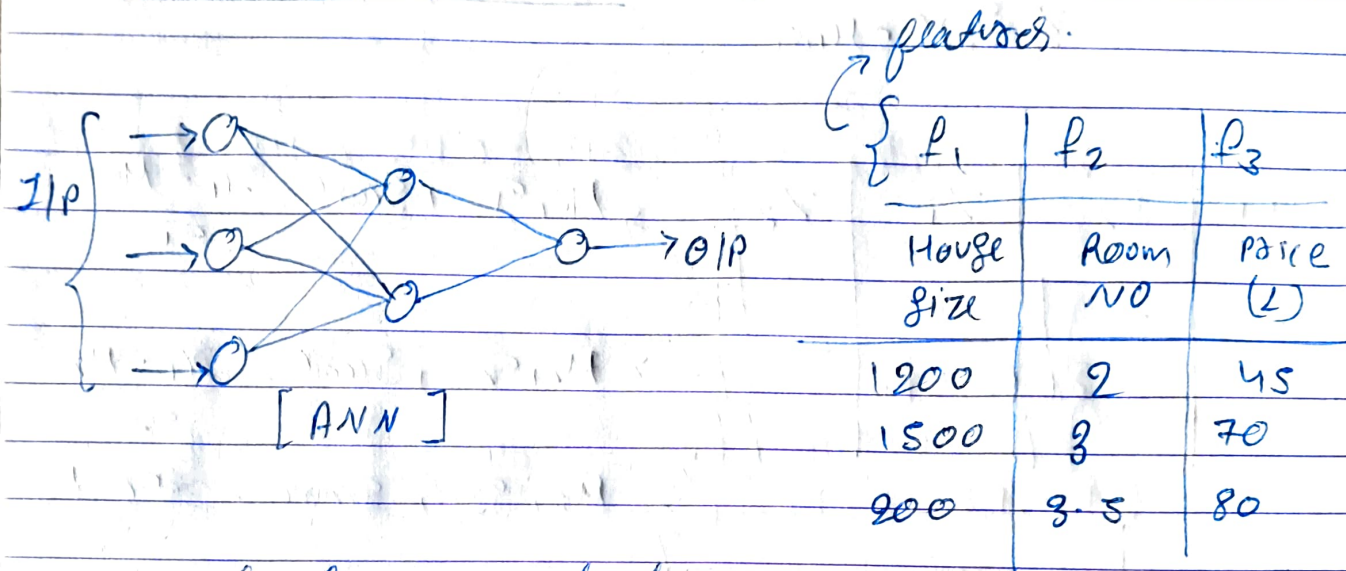


Cal

Layer Normalization In Transformer.



⇒ Batch Normalization.



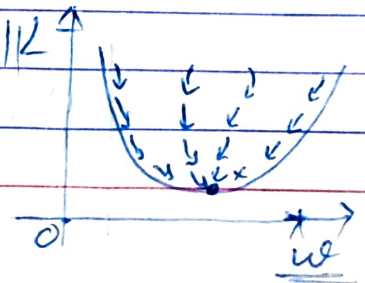
Normalization: Standard scaling. } ⇒ Z-Score ⇒ $\frac{x_i - \mu}{\sigma}$

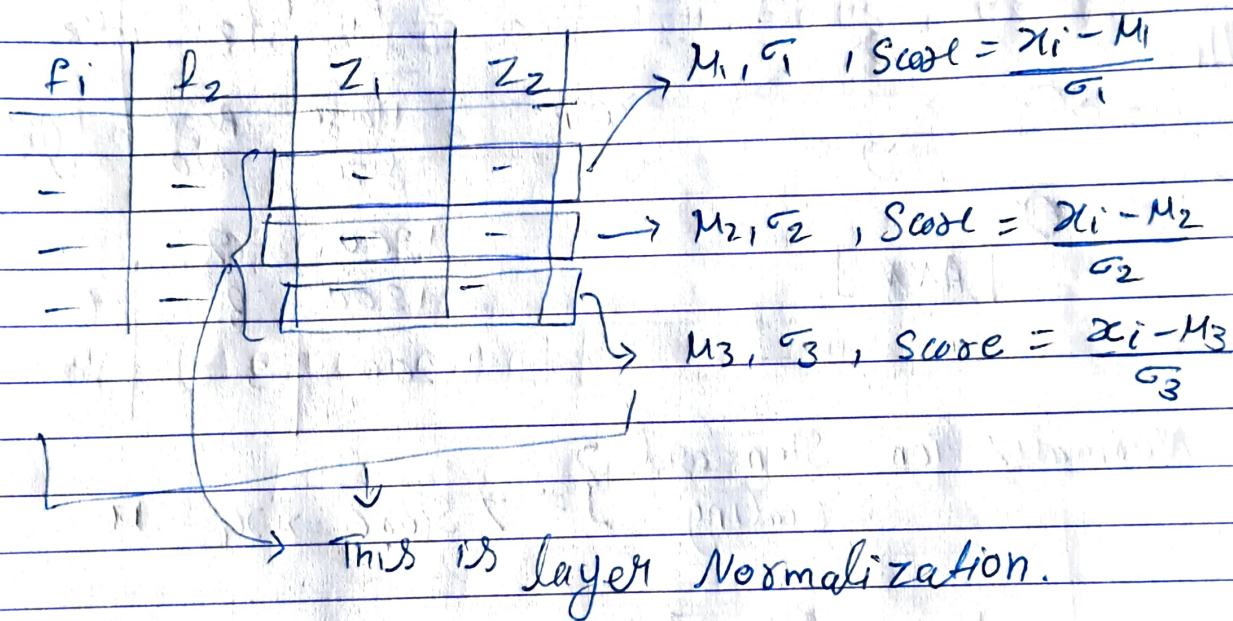
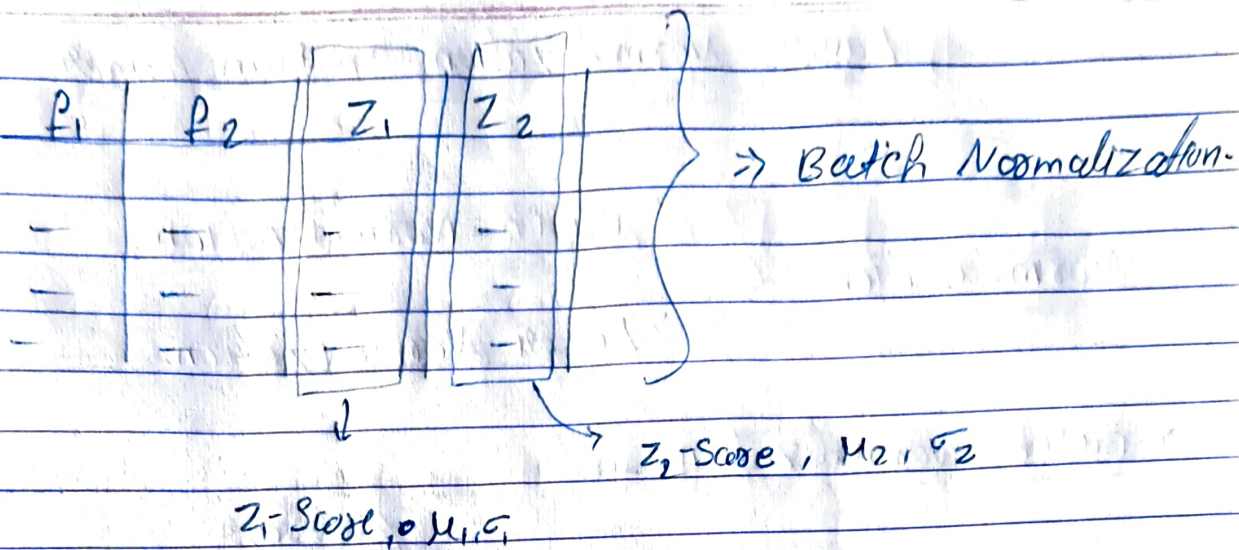
→ $\mu = 0, \sigma = 1$ → for every column.

Advantage ⇒

① Improved Training Stability.

② Faster Convergence





$\Rightarrow$ Learnable parameter.

β, γ $\nearrow$

$$z_i = \sigma(w_i^T x + b_i)$$

$$y = \gamma \left[\frac{z_i - \mu_i}{\sigma_i} \right] + \beta$$

$\rightarrow$ Scale & Shift Parameters

1) "cat" = [2.0, 4.0, 6.0, 8.0] $\rightarrow$ Vector

2) Parameters $\Rightarrow \gamma = [1.0, 1.0, 1.0, 1.0] \rightarrow$ learned scale.

$\beta = [0.0, 0.0, 0.0, 0.0]$ $\rightarrow$ shift

i) compute the mean.

$$\mu = \frac{1}{4} (2.0 + 4.0 + 6.0 + 8.0) = \frac{20.0}{4} = 5.0$$

ii) compute the variance (σ^2)

$$\sigma^2 = \frac{1}{4} \left[(2.0 - 5.0)^2 + (4.0 - 5.0)^2 + (6.0 - 5.0)^2 + (8.0 - 5.0)^2 \right]$$

$$\sigma = 5.0$$

iii) normalize the inputs.

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$\epsilon = 1 \times 10^{-5} \Rightarrow$ Avoid division by 0

$$\sqrt{\sigma^2 + \epsilon} = \sqrt{5.0 + 10^{-5}} = \sqrt{5.001} \approx 2.236$$

$$\hat{x}_1 = \frac{2.0 - 5.0}{2.236} \approx -1.34$$

$$\hat{x}_2 = \frac{4.0 - 5.0}{2.236} \approx -0.45$$

$$\hat{x}_3 = \frac{6.0 - 5.0}{2.236} \approx 0.45$$

$$\hat{x}_4 = \frac{8.0 - 5.0}{2.236} \approx 1.34$$

Normalized vector $\Rightarrow [-1.34, -0.45, 0.45, 1.34]$

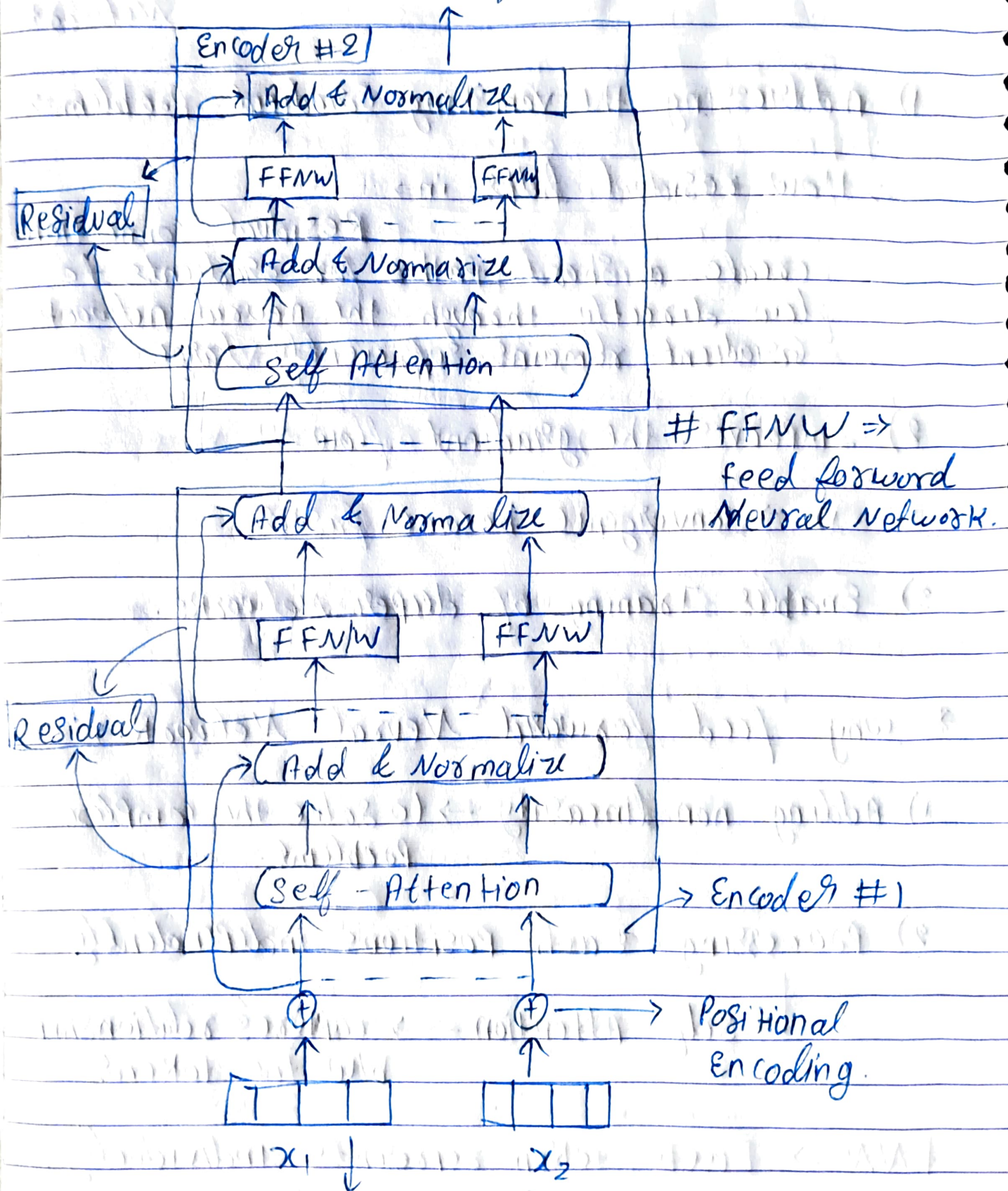
4) Scale & Shift $\Rightarrow$

$$y_i = x_i \cdot \hat{x}_i + \beta_i$$

$$y_i = ([1.0, 1.0, 1.0, 1.0] \cdot [-1.34, -0.45, 0.45, 1.34]) + [0.0, 0.0, 0.0, 0.0]$$

$$y_i = [-1.34, -0.45, 0.45, 1.34]$$

Encoder Transformer Architecture



Encoded - vector

* Residual connection $\Rightarrow$ Skip connection Neural Network.

1) Addressing the vanishing Gradient problem $\Rightarrow$

$\Rightarrow$ How residual helps in $\Rightarrow$

Residual connection create a short paths for gradients to flow directly through the neural network Gradient remains sufficiently large.

2) Improves the gradient flow $\Rightarrow$

1) $\rightarrow$ convergence become faster.

3) Enables Training of deeper Networks.

* why feed forward Neural Network $\Rightarrow$

1) Adding non linearity $\rightarrow$ to solve the complex problems.

2) processing each positions independently.

$\rightarrow$ Self-Attention $\rightarrow$ captures relationship b/w the tokens.

FNN $\Rightarrow$ Each token represents individually.

$\Downarrow$
ANN { Transforming these representations further and allows the model to learn richer representation.

3) FFN $\rightarrow$ Adds depth to the model

$\rightarrow$ depths $\uparrow\uparrow\uparrow \rightarrow$ more learning from the data.

Decoders in transformers

The transformer decoder is responsible for generating the output sequence one token at a time, using the encoder's output and the previously generated tokens.

$\Rightarrow$ 3 main components.

- ① masked multi-head self attention.
- ② multihead attention (encoder-decoder attention).
- ③ Feed forward neural network.

① masked multi-head self attention:

● there are mainly three component that we have to discuss.

① Linear projection Q, K, V .

② Scaled Dot Product Attention.

③ mask Application.

$\rightarrow$ look ahead mask
 $\rightarrow$ padded mask.

① Input Embedding and Positional Encoding.

output Embeddings:

$[0.1, 0.2, 0.3, 0.4], \rightarrow \text{Step-1}$

$[0.5, 0.6, 0.7, 0.8],$

$[0.9, 1.0, 1.1, 1.2], + P-E \Rightarrow 0$

$[0.0, 0.0, 0.0, 0.0]$

Step-2: Linear projection for Q, K, V

create query (Q), key (K) and value (V) vectors.

$$W_Q = W_K = W_V = I$$

Q = output Embedding $\times W_Q$ = output Embedding

K = output Embedding $\times W_K$ = output Embedding

V = output Embedding $\times W_V$ = output Embedding

Step-3: Scaled dot product Attention calculation.

$$\text{Score} = \frac{Q \times K^T}{\sqrt{d_K}} = \frac{Q K^T}{2} \quad \because \sqrt{d_K} = \sqrt{4} = \sqrt{2}$$

$$K^T = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}$$

$$Q_1 = [0.1, 0.2, 0.3, 0.4]$$

$$\Rightarrow \text{Score} \Rightarrow \frac{Q \cdot K^T}{\sqrt{d_K}}$$

	K_1	K_2	K_3	K_4
$Q_1 \Rightarrow [0.1, 0.2, 0.3, 0.4]$	0.1	0.5	0.9	0.0
	0.2	0.6	0.0	0.0
	0.3	0.7	1.1	0.0
	0.4	0.8	1.2	0.0

Perform the dot product with all the K 's and then calculate the Q_1 and do all this process for the other queries as well.

the scores of all Q 's are:-

$$\text{Score: } [0.3, 0.7, 1.1, 0.0] \rightarrow Q_1$$

$$[0.7, 1.0, 3.1, 0.0] \rightarrow Q_2$$

$$[1.1, 3.1, 5.1, 0.0] \rightarrow Q_3$$

$$[0.0, 0.0, 0.0, 0.0] \rightarrow Q_4$$

masked Application:

It helps to manage the structure of the sequences being processed and ensures the model behaves correctly during training and inferencing.

Reasons: ① To handle sequences of different length in batch.

- 2) To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model prediction.

Eg:- sequence-1 $\Rightarrow [1, 2, 3]$

sequence-2 $\Rightarrow [4, 5, 0]$ $\rightarrow 0$ is padding token

$\rightarrow$ influence the attention mechanism.
when the vector is large then the zero padding leads to incorrect or biased prediction.

to solve this we use the masking to ignore the zero padding.

masking $\begin{cases} \rightarrow \text{padding mask} \\ \rightarrow \text{look ahead mask} \end{cases}$

$\rightarrow$ if we take sq-1 $\rightarrow$ padding mask is: $[1, 1, 1]$

for sq-2 $\rightarrow$ padding mask is: $[1, 1, 0]$

$\rightarrow$ represent zero padding.

① Look Ahead masking $\rightarrow$

This is used to maintain the auto regressive property.

① To ensure that each position in the decoder

output sequence can only attend to the previous position, but no future position when we use the Look Ahead mask.

⑪ Sequence-to-Sequence tasks for language modeling, translation.

Eg $\rightarrow [4, 5, 0] \rightarrow [1, 1, 0] \rightarrow \text{ZeroPadding}$

Convert 1D to 2D mask.

For each token in the sequence the mask should indicate which token it can attend to.

↓

Token 1 attend to token 1, 2	1	1	0
Token 2 attend to token 1, 2	1	1	0
Token 3 attend to token 1, 2	0	0	0

Token 3 attend to token 1, 2

↓

{ based on the attention mechanism }

⑫ Look Ahead mask, decoder output.

$\begin{bmatrix} [1, 0, 0], \\ [1, 1, 0], \\ [1, 1, 1] \end{bmatrix}$ eg.

① Combine Padding And Looking Ahead mask.

→ Element wise multiplication of 2 mask.

→ combined mask →
$$\begin{bmatrix} [1, 0, 0] \\ [1, 1, 0] \\ [0, 0, 0] \end{bmatrix}$$

→ mask

~~Score~~ scores :
$$\begin{bmatrix} [0.3, 0.7, 1.1, 0.0], \\ [0.7, 1.9, 3.1, 0.0], \\ [1.1, 3.1, 5.1, 0.0], \\ [0.0, 0.0, 0.0, 0.0] \end{bmatrix}$$

Look Ahead mask →
$$\begin{bmatrix} [1, 0, 0, 0], \\ [1, 1, 0, 0], \\ [1, 1, 1, 0], \\ [1, 1, 1, 1] \end{bmatrix}$$

Padding mask →
$$\begin{bmatrix} [1, 1, 1, 0], \\ [1, 1, 1, 0], \\ [1, 1, 1, 0], \\ [0, 0, 0, 0] \end{bmatrix}$$

combined mask → Look Ahead * Padding mask.

→
$$\left. \begin{bmatrix} [1x1, 0x1, 0x1, 0x0], \\ [1x1, 1x1, 0x1, 0x0], \\ [1x1, 1x1, 1x1, 0x0], \\ [1x1, 1x1, 1x1, 1x0] \end{bmatrix} \right\} \Rightarrow \text{element wise multiplication.}$$

$$\rightarrow \left\{ \begin{array}{l} [1, 0, 0, 0], \\ [1, 1, 0, 0], \\ [1, 1, 1, 0], \\ [1, 1, 1, 0] \end{array} \right\} \rightarrow \left\{ \begin{array}{l} [1, -\infty, -\infty, -\infty], \\ [1, 1, -\infty, -\infty], \\ [1, 1, 1, -\infty], \\ [1, 1, 1, -\infty] \end{array} \right\}$$

Convert the $0 \rightarrow -\infty$

masked score $\rightarrow$

$$\left\{ \begin{array}{l} [0.3, -\infty, -\infty, -\infty], \\ [0.7, 0.9, -\infty, -\infty], \\ [1.1, 3.1, 5.1, -\infty], \\ [0.0, 0.0, 0.0, -\infty] \end{array} \right\}$$

$\Downarrow$

zero out the influence when the softmax is applied.

$\rightarrow$ softmax

$$\text{soft-score} = \text{Softmax}(\text{masked score})$$

$$\rightarrow \left\{ \begin{array}{l} [1.0, 0.0, 0.0, 0.0], \\ [0.3, 0.7, 0.0, 0.0], \\ [0.1, 0.3, 0.6, 0.0], \\ [1.0, 0.0, 0.0, 0.0] \end{array} \right\}$$

$$\rightarrow \text{because } e^{-\infty} = \frac{1}{e^{\infty}} \approx 0.0$$

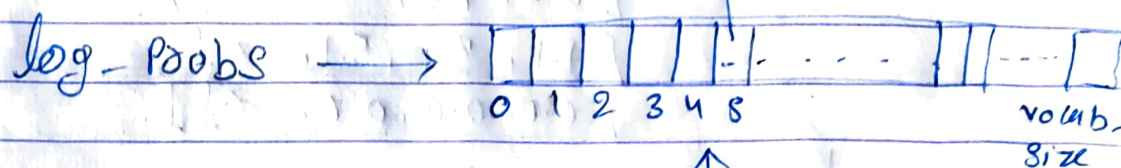
$\rightarrow$ weighted sum of values:-

$$\{ \text{Attention O/P} = \text{soft-score} \times v \}$$

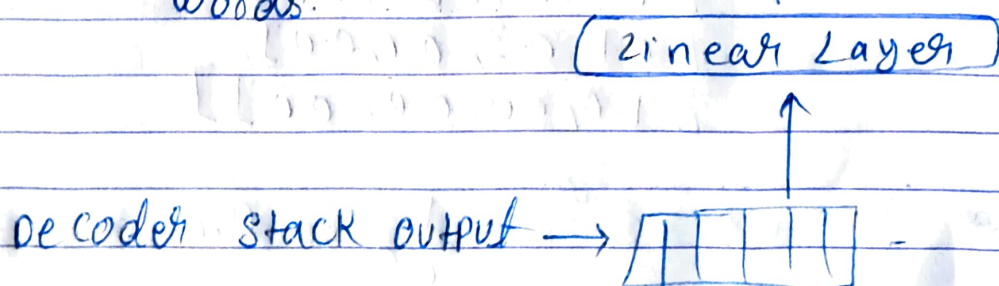
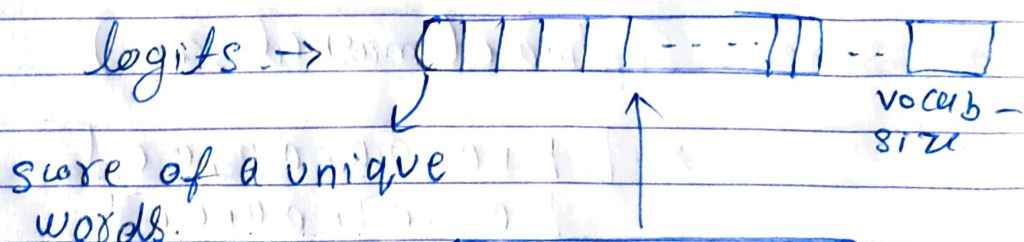
Final Linear And Softmax Layer

which word in our vocabulary
is associated with this
index.

get the index of cell with
the highest value (argmax)



multiclass → (Softmax Layer)
classification.



Linear ⇒ The Linear layer is a simple fully connected neural net that projects the vector produced by the stack of Decoder.

model $\Rightarrow$ 10,000 vocab-size $\rightarrow$ logits vector



10,000 cell size.

- ① softmax layer turns those scores into probabilities (all add up to 1.0). Then the cell with the highest probability is chosen, and the word associated with it is produced as the O/P $\Rightarrow$ with specific timestamp.